

Bug Reports

Project: AI Chatbot Management System for Facebook Messenger **Author:** Nguyễn Ngọc Minh — 22110154 **Period:** May – June 2026

Documented defects encountered during development, with reproduction steps, root cause analysis, impact assessment, and resolution status.

1. Summary

Severity	Total	Resolved	Open
● Critical	4	4	0
● Major	5	5	0
● Minor	3	3	0
● Known Limitation	5	0	5
Total	17	12	5

2. Critical Bugs

2.1. BUG-001: HTTP 500 on all API calls after server startup

Field	Detail
Severity	● Critical
Component	Backend — main.py
Status	✅ Resolved

Description: Every API call returned HTTP 500 Internal Server Error immediately after starting the FastAPI server. No endpoints were functional.

Steps to Reproduce:

1. Start uvicorn: `uvicorn main:app --reload --host 0.0.0.0 --port 8000`
2. Open browser: <http://127.0.0.1:8000/docs>
3. Call any endpoint (e.g., POST /login)
4. Receive HTTP 500 with "no such table: users"

Root Cause: The `@app.on_event("startup")` decorator was missing in main.py. Without it, `init_database()` was never called, so the SQLite tables (users, bots, messages) were never created. All subsequent queries failed with "no such table" errors.

Fix:

`python`

```
@app.on_event("startup")
def startup_event():
    init_database()
```

Impact: System completely non-functional. No login, no bot management, no webhook processing.

Lesson Learned: Always verify database initialization runs on server startup, not just on import.

2.2. BUG-002: "no such table: messages" after database init

Field	Detail
Severity	 Critical
Component	Backend — init_db.py, db.py
Status	 Resolved

Description: After fixing BUG-001, the server started but webhook processing failed with "no such table: messages" when a Facebook user sent a message.

Steps to Reproduce:

1. Start server (with BUG-001 fix applied)
2. Send a message to the Facebook Page via Messenger
3. Backend log shows: `OperationalError: no such table: messages`

Root Cause: The `init_database()` function only created the users and bots tables. The messages table CREATE statement was missing from the initialization script.

Fix: Added the messages table creation to `init_database()`:

sql

```
CREATE TABLE IF NOT EXISTS messages (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    page_id TEXT NOT NULL,
    sender_id TEXT NOT NULL,
    message TEXT NOT NULL,
    is_from_user INTEGER NOT NULL CHECK(is_from_user IN (0, 1)),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (page_id) REFERENCES bots(page_id) ON DELETE CASCADE
);
CREATE INDEX IF NOT EXISTS idx_messages_page_created
ON messages(page_id, created_at);
```

Impact: All message storage and retrieval failed. Bot could not save conversations or retrieve context.

2.3. BUG-003: BotUpdate is_active type mismatch causes toggle failure

Field	Detail
Severity	🔴 Critical
Component	Backend — schemas.py
Status	✅ Resolved

Description: Toggling a bot on/off from the Android app silently failed. The bot's is_active status did not change in the database.

Steps to Reproduce:

1. Open Android app → Bot List
2. Toggle a bot's switch from ON to OFF
3. Refresh the list — bot status unchanged
4. Backend log shows no error but UPDATE affected 0 rows

Root Cause: The BotUpdateRequest Pydantic model defined is_active as bool, but SQLite stores it as INTEGER (0 or 1). When Android sent is_active: 0, Pydantic coerced it to False, and the SQL UPDATE SET is_active = False did not match SQLite's integer comparison.

Fix:

```
python
# schemas.py
class BotUpdateRequest(BaseModel):
    is_active: int # Changed from bool to int
```

Impact: Core feature (enable/disable bot) completely broken. Admins could not control bot behavior.

2.4. BUG-004: Message created_at Serialization Crash

Field	Detail
Severity	🔴 Critical
Component	Backend — schemas.py
Status	✅ Resolved

Description: The chat history endpoint GET /bots/{id}/history returned HTTP 500 when messages existed in the database.

Steps to Reproduce:

1. Have a bot with at least one message in the database
2. Call GET /bots/{id}/history
3. Receive HTTP 500: "Object of type datetime is not JSON serializable"

Root Cause: The Message Pydantic model defined created_at as datetime, but SQLite returns timestamps as raw strings (e.g., "2026-06-01 14:51:23"). Pydantic attempted to parse the string into a datetime object, which then failed JSON serialization because FastAPI's default encoder could not handle the format.

Fix:

python

schemas.py

class Message(BaseModel):

created_at: str # Changed from datetime to str

Impact: Chat history completely inaccessible. Admins could not review any conversations.

2.5. BUG-005: java.time.LocalDate Crash on Android API 24

Field	Detail
Severity	🔴 Major
Component	Android — BotStatsActivity.kt
Status	✅ Resolved

Description: The statistics dashboard crashed immediately on launch with java.lang.NoClassDefFoundError on devices running Android 7.0 (API 24).

Steps to Reproduce:

1. Run app on emulator with API 24 (Android 7.0)
2. Navigate to Bot List → tap Statistics icon
3. App crashes immediately

Root Cause: BotStatsActivity.formatDate() used java.time.LocalDate.parse() and DayOfWeek.getDisplayName(), which require API level 26 (Android 8.0). The project's minSdk is set to 24.

Fix: Replaced all java.time calls with SimpleDateFormat:

kotlin

private fun formatDate(dateStr: String): String {

return try {

```

    val inputFmt = SimpleDateFormat("yyyy-MM-dd", Locale.getDefault())
    val date     = inputFmt.parse(dateStr) ?: return dateStr
    val dayFmt   = SimpleDateFormat("EEE", Locale("vi", "VN"))
    val dateFmt  = SimpleDateFormat("d/M", Locale.getDefault())
    "$ {dayFmt.format(date)} \n$ {dateFmt.format(date)} "
} catch (e: Exception) {
    dateStr.takeLast(5)
}
}
}

```

Impact: Statistics feature completely unusable on ~6% of Android devices in Vietnam still running API 24-25.

Lesson Learned: Always check API level requirements when using Java standard library classes. java.time is only available from API 26+.

2.6. BUG-006: Unresolved Reference getBotStats

Field	Detail
Severity	🔴 Major
Component	Android — BotViewModel.kt, BotRepository.kt, ApiService.kt
Status	✅ Resolved

Description: After adding the statistics feature to BotViewModel, the project failed to compile with 18 errors in BotViewModel.kt.

Steps to Reproduce:

1. Add getBotStats() function to BotViewModel.kt
2. Build project
3. Compilation fails: "Unresolved reference 'getBotStats'" at line 120
4. Additional cascade errors: "Cannot infer type for parameter 'T'"

Root Cause: The call chain was incomplete. BotViewModel called repository.getBotStats(), but BotRepository did not have this method, and ApiService did not define the HTTP endpoint. Kotlin could not infer types in the launch block because the called function did not exist, causing cascade type inference failures.

Fix: Added the missing methods to complete the chain:

```

ApiService.kt → @GET("/bots/{bot_id}/stats") suspend fun getBotStats(...)
BotRepository.kt → suspend fun getBotStats(token, botId):
Result<BotStatsResponse>

```

Impact: App would not compile. Statistics feature blocked entirely.

Lesson Learned: When adding a new feature, work bottom-up: API interface → Repository → ViewModel → UI.

2.7. BUG-007: Bot Does Not Reply to Facebook Messages

Field	Detail
Severity	🔴 Major
Component	Integration — Facebook Webhook + ngrok + Backend
Status	✅ Resolved

Description: After configuring the full system, sending a message to the Facebook Page produced no reply from the bot. The message appeared to be ignored completely.

Steps to Reproduce:

1. Start all services (Ollama, FastAPI, ngrok)
2. Configure webhook on Meta Developer Console
3. Send a message via Facebook Messenger to the Page
4. Wait 10+ minutes — no response

Root Cause: Multiple potential causes identified through systematic debugging:

- Webhook not subscribed to the correct Page (Page not added under Webhooks)
- messages event not ticked in webhook subscriptions
- Bot is_active set to 0 (disabled)
- Page ID in bot config did not match the actual Facebook Page ID
- ngrok tunnel expired or disconnected

Fix: Systematic checklist verification:

1. Verify ngrok URL is accessible: curl <https://xxx.ngrok-free.dev>
2. Verify backend receives webhook: check uvicorn terminal for POST /webhook logs
3. Verify Page is subscribed: Meta Console → Webhooks → Page listed with ☒ messages
4. Verify bot is active: Android app → bot status green
5. Verify Page ID matches: Facebook Page About → compare with bot config

Impact: Core functionality completely non-operational. The primary purpose of the system (auto-reply) failed silently.

Lesson Learned: Silent failures are the hardest to debug. Add logging at every stage of the webhook flow.

2.8. BUG-008: ngrok Configured on Port 80 Instead of 8000

Field	Detail
Severity	● Major
Component	Infrastructure — ngrok Configuration
Status	✓ Resolved

Description: ngrok default documentation shows ngrok http 80, but the FastAPI backend runs on port 8000. Using port 80 results in a "connection refused" error.

Steps to Reproduce:

1. Run ngrok http 80 (copying from ngrok docs)
2. Open <https://xxx.ngrok-free.dev> in browser
3. See ngrok error page: "failed to connect to localhost:80"

Root Cause: The ngrok quickstart documentation defaults to port 80 (standard HTTP). The FastAPI server is configured to run on port 8000 via `uvicorn main:app --port 8000`.

Fix:

```
bash
```

```
ngrok http 8000 # Not 80
```

Impact: Entire system unreachable from the internet. Facebook cannot deliver webhook events.

2.9. BUG-009: Gradle Error "Cannot Locate Tasks That Match ':app:testClasses'"

Field	Detail
Severity	● Major
Component	Android — Gradle Build System
Status	✓ Resolved

Description: After adding multiple new files (`BarChartView.kt`, `BotStatsActivity.kt`, `SecurePrefs.kt`), the project failed to sync with a Gradle error.

Steps to Reproduce:

1. Add several new Kotlin files to the project
2. Attempt to build
3. Error: "Cannot locate tasks that match ':app:testClasses' as task 'testClasses' not found"

Root Cause: Android Studio Gradle cache corruption after bulk file additions. The incremental build cache contained stale references.

Fix:

File → Invalidate Caches → tick all boxes → Invalidate and Restart

If persists: manually delete .gradle/ and app/build/ directories, then re-sync.

Impact: Development blocked. Cannot compile or run the app.

Lesson Learned: Treat Gradle cache corruption as a distinct debugging step. When errors appear after adding new files but the code is correct, cache is the likely culprit.

3. Minor Bugs

3.1. BUG-010: FACEBOOK_APP_SECRET warning on every webhook request

Field	Detail
Severity	● Minor
Component	Backend — main.py
Status	✅ Resolved

Description: Console displays warning on every webhook event: ⚠️ [Webhook] FACEBOOK_APP_SECRET chưa được cấu hình — bỏ qua verify signature

Steps to Reproduce:

1. Leave FACEBOOK_APP_SECRET empty in .env
2. Start the server
3. Receive any webhook request
4. Warning appears in console

Root Cause: The .env file did not contain the FACEBOOK_APP_SECRET value. The backend is designed to skip signature verification in dev mode when this value is empty.

Fix: Add the App Secret to .env:

env

FACEBOOK_APP_SECRET=your_actual_app_secret_here

Impact: Webhook functions correctly but without signature verification — security layer 1 is disabled. Acceptable for development, not for production.

3.2. BUG-011: Chat History Pagination Not Triggered

Field	Detail
Severity	● Minor
Component	Android — ChatHistoryActivity.kt
Status	✓ Resolved

Description: Pagination trigger for loading older messages never fired. Scrolling to the top of the chat history did not load more messages.

Root Cause: layoutManager was declared as a generic RecyclerView.LayoutManager instead of LinearLayoutManager. The findFirstVisibleItemPosition() method is specific to LinearLayoutManager and returns -1 on the base class.

Fix:

kotlin

```
private val layoutManager by lazy {  
    LinearLayoutManager(this)  
}
```

Impact: Users could only see the most recent messages. Older messages were inaccessible.

3.3. BUG-012: CreateBotDialog uses CoroutineScope instead of lifecycleScope

Field	Detail
Severity	● Minor
Component	Android — CreateBotDialog.kt
Status	✓ Resolved

Description: Potential memory leak when the dialog is dismissed while a network request is in progress.

Root Cause: The dialog used CoroutineScope(Dispatchers.Main) which is not tied to the dialog's lifecycle. If the dialog is dismissed during an API call, the coroutine continues running with a reference to the destroyed dialog.

Fix: Replaced with lifecycleScope which automatically cancels coroutines when the dialog is destroyed:

kotlin

```
lifecycleScope.launch {  
    // network call  
}
```

Impact: Potential memory leak and crash if dialog is dismissed during network call. Low frequency but poor practice.